

datasets/TUS_complete.py

功能说明与修改记录 (2026-06-13)

一、文件定位

TUS_complete 是项目中用于加载真实跟踪超声 (Tracked Ultrasound) 扫描数据的 PyTorch Dataset 类, 继承自 BaseDataset。数据源为 HDF5 文件, 按 root/<subject>/<scan>.h5 组织, 每个文件包含 frames (图像帧序列, uint8) 和 tforms (每帧对应的 4×4 跟踪变换矩阵, float32)。配置文件路径: res/datasets/TUS_complete.json。

二、核心方法说明

2.1 more(cfg) — 静态配置补全

- 解析 h5 和 calib 路径 (转为绝对路径)
- 计算 source.elements = width $\times$ height $\times$ channel (供外部模型配置使用)
- 设置 num_workers=0、pin_memory=False、device (CUDA 优先)
- 若配置文件未指定 train_val_test_split, 默认 [0.6, 0.2, 0.2]

2.2 _build_series(tforms_np, H_img, W_img) — 构建世界坐标锚点序列

将跟踪仪输出的每帧 4×4 变换矩阵转换为世界坐标系下的三维锚点序列, 输出形状为 (N, 3, 3), 三个锚点分别为: 图像中心 (W/2, H/2)、左下角 (1, H)、右下角 (W, H)。变换流程: 帧间变换 $\rightarrow$ 标定矩阵合并 (tform_calib_R_T 逆 $\times$ tforms_f2f0 $\times$ tform_calib) $\rightarrow$ 像素坐标投影到世界坐标。

2.3 load() — 数据全量加载与划分

- 按 subject 目录进行 train/val/test 三段划分 (避免数据泄漏)
- 读取每个 .h5 文件的全部帧 (frames 全量) 和变换矩阵 (tforms)
- frames 若与配置尺寸不符则双线性插值 resize 到 (H_out, W_out)
- 调用 _build_series 构建锚点序列, 再调用 preprocessing 得到 target_point 和 target_dof
- 将实际扫描文件数写回 cfg.train_test_range, 供 BaseDataset.split() 使用
- data_count = $\sum$ (各集合文件数 $\times$ series_per_data[i]), 控制每个 epoch 的采样总量

2.4 preprocessing(tp) — 坐标归一化与 DOF 计算

输入: 锚点序列 tp, 形状 (N $\times$ 9) 或 (N, 3, 3)。(1) 将序列连同其镜像扩展点合并, 求所有点的逐维最小值 min_loca; (2) 将整段序列平移, 使最小坐标为零 (局部归一化); (3) 调用 utils.simulation.series_to_dof 计算相邻帧间的 DOF (自由度向量); (4) 返回平铺后的 tp (N, 9) 与 td (N, dof_dim)。

2.5 pad(data, max_length) — 序列截断

若序列长度超过 max_length 则截断, 否则原样返回。设计与 DDH/Fetus 数据集的 pad()

方法一致，用于将变长序列统一为固定长度 `seq_len`，保证 `DataLoader` 默认 `collate_fn` 在 `batch_size > 1` 时不报错。

2.6 __getitem__(index) — 样本采样

每次调用的完整处理流程：

```
1. get_idx(index)    -> idx (扫描索引)
2. source[:,frame_rate] -> 以随机帧率降采样全量帧序列
3. target_point[:,frame_rate]
4. info = len(source) -> 记录截断前的真实帧数
5. pad(source, seq_len) -> 截断到固定长度
6. pad(target_point, seq_len)
7. preprocessing(target_point) -> target_point(seq_len,9), target_dof
8. get_optical_flow(source) -> optical_flow
9. get_edge(source) -> edge
10. 组装 sample_dict 返回
```

`sample_dict` 包含：`source` (`seq_len,1,H,W`)、`target` (`seq_len, dof+9`)、`optical_flow`、`edge`、`frame_rate`、`info` (原始长度)。

三、今日修改记录 (2026-06-13)

修改一：放弃懒加载，改为全量加载

原因：懒加载（按需读取 2 帧）阻碍 LSTM 获取完整时序信息，且 `h5py` 多进程读取存在锁竞争风险。

位置	修改前	修改后
<code>load()</code>	仅存路径与帧数字字典	读取全量 frames/tforms 为 tensor
<code>__getitem__</code>	<code>h5py</code> 按需读 2 帧	直接索引内存中的 tensor
<code>import</code>	<code>import random</code>	已移除（不再使用）

修改二：修复 `frame_rate` 未生效问题

原始代码采样了 `frame_rate` 但固定取相邻两帧 `[n0, n0+1]`，`frame_rate` 对帧选择无任何影响。

```
# 修改前
frame_rate = torch.randint(cfg.frame_rate[0], cfg.frame_rate[1]+1, (1,))
n0 = random.randint(0, N-2)
idx_frames = [n0, n0+1] # frame_rate 无效

# 修改后
frame_rate = torch.randint(cfg.frame_rate[0], cfg.frame_rate[1]+1, (1,))
source = source[:,frame_rate.item()] # 步长采样全序列
target_point = target_point[:,frame_rate.item()] # 同步
```

修改三：引入 `seq_len` 固定输出序列长度

问题：`[:,frame_rate]` 在 `frame_rate` 随机时输出变长序列 (`N//fr` 帧)，`DataLoader` 默认 `collate_fn` 无法将不同长度的张量堆叠为批次，`batch_size>1` 时报错。同时变长序列也阻碍 LSTM

训练（无法保证每次输入的时序上下文长度一致）。

解决方案：参考 DDH/Fetus 数据集的 `pad()` 截断机制，在 `frame_rate` 降采样后将序列截断至固定长度 `seq_len`。

```
# 新增方法
def pad(self, data, max_length):
    if len(data) > max_length:
        return data[:max_length]
    return data

# __getitem__ 中新增
info = torch.tensor([len(source)]) # 截断前记录真实长度
source = self.pad(source, cfg.seq_len) # 截断
target_point = self.pad(target_point, cfg.seq_len)
```

配置文件 `res/datasets/TUS_complete.json` 同步新增字段：`"seq_len": 10`

`seq_len=10` 的依据：`frame_rate` 最大值为 8，实际扫描通常含数百帧， $N/8 >> 10$ ，截断始终可触发，输出长度严格固定。

四、修改前后对比

特性	旧版 TUS_complete	修改后	对比参照
加载方式	懒加载（按需读 2 帧）	全量加载到内存	与 TUS_subject 一致
内存占用	极低（仅路径）	较高（全量 tensor）	TUS_subject 同级
数据划分	Subject 级（无泄漏）	Subject 级（无泄漏）	保持不变
frame_rate	采样但无效	步长采样全序列	与 TUS.py 一致
输出帧数	固定 2 帧	固定 <code>seq_len</code> 帧	与 DDH/Fetus 一致
LSTM 支持	[否]（单时间步）	[是]（ <code>seq_len</code> 时间步）	目标特性
批次合并	[否]（变长， $\text{batch} > 1$ 崩溃）	[是]（定长，正常）	解决问题一
info 含义	截断后长度（恒为 2）	截断前真实帧数	与 DDH/Fetus 一致

五、遗留注意事项

- `target_dof_list` 在 `load()` 中计算并存储，但 `__getitem__` 从不读取它（每次对子序列重新计算）。这导致全量数据常驻内存但毫无用途，可考虑删除以节省内存。
- 若某扫描帧数 N 在 `frame_rate` 最大值下满足 $N/\text{frame_rate_max} < \text{seq_len}$ ，`pad()` 将返回短于 `seq_len` 的序列，仍会导致 `collate_fn` 报错。建议在 `more()` 或 `load()` 中校验扫描帧数下限。
- `Spine.py` 存在相同的变长序列问题，且 `source[:,frame_rate]` 使用 `shape=(1,)` tensor 作为 slice step，在严格的 PyTorch 版本中会抛 `TypeError`，应改为 `source[:,frame_rate.item()]`。